TAKE-HOME ASSIGNMENT

Project SLATE — AI/ML Intern Take-Home Assignment

> **Build an AI Canvas. Then prove what it costs.**

Chat boxes flatten thinking into a single column. Your job is to build a surface where handwriting, sketches, equations and spatial layout become the prompt — and to instrument it so that every millisecond and every token is measured, attributed and defensible.

We are not asking for a big product. We are asking for one small product built to an unreasonable standard.

Role	AI/ML Intern
Organisation	Scholera
Due	See the email that sent you this assignment
Effort	~10 days, part-time
Version	1.0

> **Reference project** (open source — read it, do not clone it): github.com/penecho/penecho · Licensed AGPL-3.0. Section 2 explains exactly what borrowing is allowed.

---

01 · What you are building

The brief in one page. Read this twice before writing any code.

Build a **web-based AI canvas**: an infinite two-dimensional workspace where a person can write, sketch and arrange ideas with a pen, stylus, mouse or finger. When they pause, the region they were working in is sent to a multimodal language model along with its spatial context. The model's answer comes back **as an object on the canvas**, placed beside the work that produced it — not in a sidebar, not in a chat log.

Around that loop you will build a **measurement layer** that records the latency and token cost of every single model call, exposes them live, and lets you make and defend an engineering trade-off with numbers rather than opinions.

> **The one rule that governs everything else**
>
> **Depth beats breadth. Every time.** A canvas that does four things flawlessly — with clean input handling, honest error states, sub-second interaction and a metrics panel you'd show a CFO — scores far higher than one that does fifteen things at 60%. If you find yourself adding a feature because it sounds

> impressive, delete it and go polish something instead.

What we are actually assessing

Product judgement	Can you tell the difference between a feature that matters and a feature that demos well?
Systems thinking	Multimodal LLM calls are slow, expensive and non-deterministic. Does your architecture accept that reality or fight it?
Measurement literacy	Can you instrument a system, choose the right statistics, and reach a conclusion you can defend?
Craft	Does the thing feel good in the hand? Does it fail gracefully? Would you put your name on it?
Originality	Given a reference implementation, do you copy it or do you think past it?

> **Deliberately not assessed**
>
> Training or fine-tuning models. Custom OCR or handwriting-recognition models — a modern multimodal model already reads handwriting; your job is the system around it, not the recogniser. Visual design awards. Deployment to a cloud provider. Scale beyond a single user.

---

02 · The reference project

Study it hard. Then close the tab and build your own.

> **github.com/penecho/penecho** PenEcho — *"Think with AI beyond the chat box."* An open-source shared canvas for handwriting, equations, diagrams and spatial reasoning. Node.js, browser front end, pluggable model executors. Licensed GNU AGPL-3.0.

How to use it

- **Run it first.** Spend an evening actually using it. Note every moment it felt slow, confusing, magical or broken. Those notes are the raw material for Section 6.
- **Read the architecture notes** at `docs/architecture.md` and the README section *How it works*. Understand why it sends a cropped region plus geometry rather than the whole canvas.
- **Read its open issues and discussions.** The gaps a project admits to are usually the most interesting problems in it.
- **Steal ideas, not implementations.** *"A returned answer should be a movable draft you accept or discard"* is an idea, and a good one. Their draft-object class is an implementation.

> **Hard boundary**
>
> **Do not fork, clone, vendor or copy-paste PenEcho's source into your submission. We will diff.** Substantial copied code is an automatic disqualification, and it also creates a licensing problem: AGPL-3.0 is copyleft, so derivative work must itself be released under AGPL.
>
> If you deliberately reimplement a specific idea from it, say so in `ATTRIBUTION.md` — one line per borrowed idea. **Honest attribution earns marks. Silent copying loses all of them.**

Other things worth reading

tldraw / excalidraw	Two mature open-source canvases. Look at how they model strokes, handle pointer events, and keep pan/zoom smooth.
W3C Pointer Events	w3.org/TR/pointerevents — coalesced events, pressure, tilt. If you are using mouse events in 2026, you are dropping input.
Provider token docs	Your chosen provider's documentation on how image inputs become tokens. You will need this for Section 5.
OpenTelemetry	opentelemetry.io/docs — you don't have to use it, but read how spans and attributes are modelled before inventing your own tracing format.

---

03 · Scope

What is in, what is out, and why the "out" list is the more important one.

PenEcho ships a wide feature surface: plugins, diagram renderers, animation scenes, photo search, LAN sharing, desktop packaging. You have ten part-time days. Attempting that breadth guarantees a shallow submission. So we are fixing the scope for you.

✅ **In scope — build all of this**	❌ **Out of scope — do not build**
The canvas core loop (Section 4)	User accounts, auth, teams, real-time collaboration
The metrics & observability layer (Section 5)	A plugin system or extension marketplace
A measured optimisation experiment (Section 5)	Desktop packaging or mobile apps
A written feature-ideation study (Section 6)	Cloud deployment, Docker orchestration, CI/CD
Exactly **one** original feature of your own, shipped to the same quality bar as the core	A second or third original feature
	Your own handwriting-recognition model

> **Why the "out" list is scored**
>
> Building something explicitly out of scope does not earn bonus points — **it costs them**, under *Scope discipline* in the rubric. Knowing what not to build is the single most valuable habit an engineer can bring to a small team. This assignment tests it directly.

The quality bar, stated concretely

"Perfection" is not a vibe. For the core loop we will check for these specific things:

Input fidelity	No dropped or jittery points at speed. Pressure and tilt used if the device reports them. Palm/touch behaviour is deliberate, not accidental.
Interaction latency	Pan, zoom and draw stay responsive with 5,000+ strokes on the canvas. State your measured frame timing in the README.
Reversibility	Undo/redo covers everything a user can do — including accepting an AI draft. Nothing is destroyed without a way back.
Honest failure	Timeouts, rate limits, malformed model output and offline states each have a specific, non-blocking UI response. No silent failures. No infinite spinners.
Keyboard	Every frequent action has a shortcut, and they are discoverable.
Cold start	A stranger clones your repo and reaches a working canvas in under five minutes using only the README.

---

04 · Part A — The canvas core loop

Six requirements. All mandatory. This is the spine of the submission.

A1 · A surface that scales

A logically large canvas (PenEcho uses 20,000 × 20,000) with smooth pan and zoom. **Do not allocate a bitmap for the whole thing** — use sparse tiling, a scene graph, or another approach you can justify. Explain your choice and its memory profile in the README.

A2 · Ink that feels like ink

Capture strokes via Pointer Events with **coalesced-event handling**. Store a structured stroke model, not a flat image. Support at minimum: draw, erase, colour, lasso or marquee selection, move, resize, delete, undo/redo.

A3 · Context extraction

This is the intellectually interesting part and it is where most submissions will separate. When a request fires, you must decide what to send. Sending the whole canvas is wasteful and slow; sending too little loses meaning. Build a deliberate strategy and document it:

- How do you determine the region of interest — the selection, the recent-ink bounding box, a spatial cluster?
- What margin do you add, and why? What resolution do you rasterise to, and how did you pick it?
- What non-image context travels with it — coordinates, zoom level, nearby object types, prior accepted drafts?
- What triggers a request — an explicit gesture, an idle timer, both? What is the default idle delay and why?

A4 · Answers as canvas objects

The model's response returns as a **draft object** placed near its source region: movable, resizable, copyable, and explicitly **accepted or discarded** before it becomes part of the document. Confirmed ink and unconfirmed drafts must never be confused with each other, visually or in the data model. Render at least Markdown and LaTeX; more is optional.

A5 · Persistence and export

Save and reload a canvas (a documented JSON format is fine). Export confirmed content to PNG with sane cropping. **Unconfirmed drafts are never included in a save or an export.**

A6 · Latency-aware UX

A multimodal call can take fifteen seconds. Your interface must stay alive throughout:

- An **immediate, in-place placeholder** anchored to the region — the user must never wonder whether it registered.
- **Streaming render** if the provider supports it; a determinate or informative progress state if not.
- The canvas **remains fully interactive** during a request. Drawing while waiting is normal, not an edge case.
- A **user-visible cancel.** A new request supersedes an in-flight one; the superseded call is aborted, not orphaned — and **its cost is still recorded** (see WTR in Section 5).

> **Sharpen your own edge**
>
> Nothing above dictates how the canvas should look or feel. Modes, themes, gestures, the shape of the draft object, how a request is invoked — these are yours. We would rather see one opinionated interaction you can argue for than a careful imitation of the reference.

---

05 · Part B — The metrics layer

Latency and token accounting. ***The highest-weighted section of this assignment.***

Every model call must pass through an instrumentation layer that records timing and token usage to a structured trace. **Instrumentation added as an afterthought is obvious and scores badly** — design it as a wrapper around your model client from day one.

B1 · Latency: measure these six segments

A single end-to-end number tells you a request was slow. It does not tell you *where*.

Symbol	Segment	Starts	Ends
`t_capture`	Capture & encode	Trigger fires	Payload encoded
`t_dispatch`	Client & server overhead	Payload encoded	Provider request sent
`ttfb`	Time to first byte	Provider request sent	First byte received
`ttft`	Time to first token	Provider request sent	First content token
`t_stream`	Generation	First content token	Last content token
`t_render`	Paint	Last content token	Draft visible on canvas
`e2e`	**End to end**	Trigger fires	Draft visible on canvas

> **Statistics requirement**
>
> Report **p50, p90, p95, p99, max and n** for every latency segment. LLM latency distributions have long right tails; a mean hides exactly the failures your users remember. **A report that leads with average latency loses marks in this section regardless of how good the code is.**

B2 · Tokens: capture what the provider reports, estimate the rest

Record per request: `input_text_tokens`, `input_image_tokens`, `output_tokens`, `reasoning_tokens` (many providers bill hidden thinking tokens as output — find out whether yours does), `cache_read_tokens`, and `total_tokens`.

Where the provider does not report image tokens separately, implement an estimator from the provider's documented tiling rule, state its assumptions, and validate it against reported totals across **at least twenty requests**. Report your estimator's mean absolute error. *An honest estimator with a known error bar is worth more than a confident guess.*

Cost is computed from a rate table held **in configuration — never hard-coded** — as `(in × rate_in + out × rate_out + reasoning × rate_out) ÷ 1,000,000`, with the rate source and date cited in `METRICS.md`.

B3 · Four derived KPIs you must implement

KPI	Definition and why it exists
CPAD — Cost per Accepted Draft	Total spend ÷ drafts the user accepted. Raw cost-per-request rewards cheap, useless answers. This one does not.
DAR — Draft Acceptance Rate	Accepted drafts ÷ drafts returned. Your only in-app signal of answer quality. Instrument accept and discard from the start.
WTR — Wasted Token Ratio	Tokens spent on discarded, cancelled, superseded, timed-out or errored requests ÷ all tokens. Measures how much your trigger policy costs the user for nothing.
BC — Budget Compliance	Share of requests meeting a latency budget **you declare in the README** (e.g. p95 e2e ≤ 8s). Declaring a target and measuring against it is the entire discipline.

B4 · Two surfaces: a live panel and a durable trace

- **An in-app metrics panel** — toggleable, showing the last request's segment breakdown, session totals, running cost, and the four KPIs. It must update live, and it must not itself cause jank.
- **A JSONL trace file**, one line per request, credential-redacted, with a documented schema. **Ship at least 50 real trace lines** with your submission.

Minimum trace schema — extend it, don't shrink it:

```
// one JSON object per line, appended atomically
{
  "request_id": "req_01JGX7...",        // stable, sortable
  "session_id": "ses_01JGX6...",
  "ts_start":   "2026-08-04T11:03:22.481Z",
  "trigger":    "idle_pause",           // idle_pause | explicit | refine
  "provider":   "anthropic",
  "model":      "claude-sonnet-4-5",
  "effort":     "medium",
  "config_id":  "cfg_B_webp_1024",      // ties this run to an experiment arm
  "input": {
    "crop_px": [1024, 768], "format": "webp", "bytes": 48211,
    "zoom": 1.0, "stroke_count": 37, "prompt_chars": 812
  },
  "latency_ms": {
    "t_capture": 61, "t_dispatch": 18, "ttfb": 940,
    "ttft": 1180, "t_stream": 3420, "t_render": 44, "e2e": 5663
  },
  "tokens": {
    "input_text": 214, "input_image": 1105, "input_image_source": "reported",
    "output": 388, "reasoning": 1024, "cache_read": 0, "total": 2731
  },
  "cost_usd": 0.02614,
  "outcome":  "accepted",               // accepted | discarded | cancelled
                                        // | superseded | timeout | error
  "error":    null,
  "retries":  0
}
```

B5 · The experiment

Instrumentation exists to answer questions. Run a real one and write it up in `REPORT.md`.

> **Protocol**
>
> **1. Fix your benchmark.** Build **The Five Canvases** — five saved canvas files you reuse for every run, committed to the repo:
>
> 1. A handwritten multi-line equation
> 2. A rough boxes-and-arrows system sketch
> 3. A handwritten plain-language question
> 4. A dense canvas — 300+ strokes, mixed content
> 5. A deliberately ambiguous or half-erased scrawl

> **2. Pick one variable and define at least three arms.** Good candidates: image format (WebP vs PNG), crop resolution (512 / 1024 / 1536 px), reasoning effort, model tier, prompt length, or a cheap-model-first routing tier.
>
> **3. Run every arm across all five canvases, at least three repetitions each** (≥45 requests per arm). **Randomise or interleave arm order** — do not run all of arm A then all of arm B, or provider-side variance will contaminate your result.
>
> **4. Report** a table of p50 / p95 e2e, mean tokens, mean cost and DAR per arm; at least one chart; and a clear recommendation with its trade-off stated plainly.

> **Then act on it**
>
> Implement **at least two optimisations** in your product and measure each one before and after with the same protocol. Report the p50 and p95 latency delta, the cost delta, and any quality cost. **An optimisation you cannot show a number for does not count.** A negative result, honestly reported — *"I expected this to help, it did not, here is the data and my theory why"* — scores full marks.

---

06 · Part C — Feature ideation, and the one you ship

Think broadly on paper. Then build narrowly, and build it properly.

The reference project's feature set is early and deliberately limited. The most valuable thing you can show us is not more code — it is evidence that you can see past an existing product and reason about what it should become.

C1 · Write `IDEAS.md`

Propose **eight to twelve features** that do not exist in the reference project today. For each, in no more than 120 words:

Field	Meaning
`problem`	The specific user moment it fixes. Not a capability — a *moment*.
`why_canvas`	Why this is better on a spatial canvas than in a chat window. **If it isn't, cut the idea.**
`model_dependency`	What the model must actually be good at for this to work at all.
`cost_class`	cheap / moderate / expensive, in tokens and latency — justified from Section 5's numbers, not vibes.
`risk`	The most likely reason this fails in real use.

> **Axes to think along — not a menu to pick from**
>
> **Memory & continuity** — the canvas remembers what you worked out last week. **Verification** — the model checks your algebra rather than answering it. **Time** — replay, branch, or diff a region's history. **Pedagogy** — hints that escalate instead of solutions that end thinking. **Cost as UX** — the user sees and steers what a request will spend. **Accessibility** — spatial thinking for people who cannot see the canvas. **Locality** — a small on-device model handling the 80% case. **Agency** — canvas objects that recompute themselves when their inputs change.

> **Two ideas are off-limits**
>
> You may not ship *"export to PDF"* or *"a chat sidebar"* as your original feature. They are listed purely so you know what the low bar looks like. Anything describable as "the same thing, but also in a panel" will not be scored as original.

C2 · Choose one. Justify it. Build it.

Score your ideas on impact, effort and running cost. Pick **exactly one** and write a paragraph on why it beat the others — **including which idea you most wanted to build and why you didn't.** Then ship it at the same quality bar as the core loop: instrumented through the same metrics layer, with its own error states, undo support and keyboard access.

> **Scoring note**
>
> **One feature at 95% scores full marks. Three features at 60% score roughly half**, and additionally lose points under *Scope discipline*. We mean this literally. It is the single most common way strong candidates lose this assignment.

---

07 · Technical constraints

Few rules, firmly held.

Constraint	Detail
Stack	Your choice, entirely. TypeScript / Python are common; anything you can defend is fine. Choose what lets you go deep in ten days, not what looks best on a CV.
Model access	Any multimodal provider, or a local model. Keys come from **environment variables only**. Ship a `.env.example`. **A committed key is an automatic fail — we scan for it.**
Provider abstraction	Not required, but a clean adapter interface that lets you swap providers is a strong signal — and it makes the Section 5 experiment far easier to run.
Runs locally	Clean clone → working canvas in ≤ 3 commands. No paid infrastructure, no cloud account, no manual database setup.
Tests	We are not counting coverage. We want tests on the parts that would **silently** break: token accounting arithmetic, cost calculation, trace-schema validity, stroke serialisation round-trip, region-extraction geometry.
Git history	Incremental, readable commits. A single "initial commit" containing everything is treated as a provenance red flag and will be questioned in the review call.
Privacy	Traces must never contain credentials. If your traces include canvas imagery, that must be **off by default** and documented.

> ### On AI coding assistants
>
> **Use them.** We do, and we would find it strange if you didn't. But two conditions apply, without exception.
>
> **One:** keep a short `AI_USAGE.md` — which tools, for which parts, and one honest paragraph on where they helped and where they led you wrong.
>
> **Two:** in the review call you must be able to explain any line in your repository, justify the architecture, and make a live change we ask for. **Code you cannot explain is worth less than code you did not write.**

Getting stuck. If you are blocked on API credits, hardware, or a stylus for testing, reply to the email that sent you this assignment, or write to **proscio@scholera-inc.com**. We would rather help than read a submission quietly compromised by a solvable constraint. Where this document is ambiguous, ask; we answer publicly to all candidates.

---

08 · How you will be scored

Out of 100, plus a capped bonus. ***Nothing here is secret.***

Criterion	What earns the marks	Pts
Canvas core loop & craft	All six A-requirements met; input fidelity; smoothness under load; reversibility; honest failure states; how it feels in the hand.	25
Metrics layer	Segment-level latency; complete token and cost accounting; the four KPIs; live panel; valid, documented, redacted traces.	22
Experiment & optimisation	Sound protocol; correct statistics; two measured optimisations with before/after deltas; a defensible recommendation.	15
Feature ideation	Depth and originality of `IDEAS.md`; quality of the cost/impact reasoning; honesty of the selection argument.	10
The one shipped feature	Genuinely original; fully finished; instrumented like everything else; earns its place on a canvas.	10
Engineering craft	Architecture, naming, error handling, meaningful tests, README quality, git history.	8
Scope discipline	You built what was asked, refused what wasn't, and can say why. Out-of-scope work is penalised here.	5
Integrity & attribution	Honest `ATTRIBUTION.md` and `AI_USAGE.md`; no copied source; clean provenance.	5
Total		**100**

> ### Bonus — capped at +10, only if the core is already excellent
>
> Meaningful accessibility work · a clean multi-provider adapter with a comparison run · a local/on-device model path with measured trade-offs · offline-first behaviour · prompt or context caching with proven savings · a genuinely novel pen interaction.
>
> **Bonus is not awarded to a submission whose core scores below 70. Polish first, always.**

> ### Instant disqualifiers
>
> - Substantial copied source from PenEcho or any other project, undeclared.
> - Credentials committed to the repository.
> - Fabricated metrics — numbers in `REPORT.md` that no trace file supports. **We check.**
> - Inability to explain your own submission in the review call.

---

09 · Deliverables

Eight items. ***A missing one is a zero for its criterion, not a deduction.***

Item	Contents
1 · repository	Public. If it must be private, email us and we'll send you the accounts to invite. Full history either way.
2 · `README.md`	Setup in ≤3 commands, architecture diagram, your context-extraction strategy, your declared latency budget, measured interaction frame timing, known limitations. *Limitations you name yourself are never held against you.*
3 · `METRICS.md`	Trace schema, segment definitions, image-token estimator and its validated error, rate table with source and date, KPI formulas, panel screenshot.
4 · `REPORT.md`	The experiment: protocol, arms, p50/p95 tables, ≥1 chart, the two optimisations with before/after deltas, recommendation and trade-off.
5 · `IDEAS.md`	8–12 features to the Section 6 template, scoring, and the selection argument.
6 · `traces/`	≥50 real, redacted JSONL trace lines, plus the five benchmark canvas files.
7 · attribution	`ATTRIBUTION.md` and `AI_USAGE.md`.
8 · demo video	Maximum 3 minutes, unlisted link. Show the core loop end to end, the metrics panel live, and your one feature. Do not narrate your setup process. Screen recording with voice is fine; production value is not scored.

---

10 · Mode of submission

One email. Follow the format exactly — it is the first thing we check.

Step 1 — Email. Send one email to **proscio@scholera-inc.com** — or simply reply to the email that sent you this assignment — with:

Subject	`SLATE – <Full Name> – <College or Organisation>`
Body	Repository link · demo video link · the model and provider you used · your three headline numbers (p50 e2e, p95 e2e, CPAD) · and one short paragraph: *"With two more weeks, here is what I would build next and why."* Keep the whole email under 250 words.
Attachments	None. Links only. **Zipped repositories will not be reviewed.**

> ### Deadline
>
> Your deadline is in the email that sent you this assignment. The commit timestamp at that point is what we review — later commits are ignored, so **tag your submission** (`git tag submission`). If something serious happens in your life, tell us before the deadline rather than after; we are reasonable people and we will work with you.

Step 2 — The review call. Shortlisted candidates get a **45-minute call** within a week of the deadline: a live demo by you (10 min), a code walkthrough where **we** pick the files (20 min), one small live change we specify (10 min), and your questions for us (5 min). Prepare to defend your **context-extraction strategy**

and your **experiment's conclusion** — those are always the two hardest questions we ask.

---

11 · A suggested ten days

Advisory, not mandatory. But note where the metrics layer sits.

Days	Focus	Aim to end with
1	Explore & decide	Reference project run and notes taken; stack chosen; context-extraction strategy sketched on paper.
2–3	Canvas foundation	Pan, zoom, ink, selection, undo. Smooth with thousands of strokes. No AI yet.
4	First model call	Region → model → draft object on canvas. Ugly is fine. The loop closes.
5	Metrics layer	Instrumentation wrapper, trace file, live panel. **Do not push this later.**
6	Harden the loop	Streaming, cancellation, supersede, error states, persistence, export.
7	Ideation & experiment	`IDEAS.md` written and scored; five benchmark canvases built; experiment running.
8	Your one feature	Built and instrumented.
9	Optimise & measure	Two optimisations shipped, before/after numbers captured.
10	Write & record	All docs finished, video recorded, cold-start tested on a clean clone.

---

12 · Questions we expect

Read these before emailing. Anything else, ask — we answer publicly to all candidates.

Can I use a canvas library like tldraw, Konva, Fabric or PixiJS? Yes, and say so in the README. Rendering primitives are not what we are testing. But if the library also does your selection, geometry and undo, you have removed a chunk of what we score — so go deeper on the metrics layer and your one feature to compensate.

Can I use PenEcho's prompts? No. Prompt design is part of context extraction, which is part of the assessment. Write your own and explain the reasoning in your README.

What if I run out of API credits? Get in touch before you are blocked. A small local multimodal model is also a perfectly acceptable path — and it makes for a more interesting Section 5 experiment than most.

Do I need a stylus or tablet? No. A mouse or trackpad is fine to build and demo with. If you have pressure-sensitive hardware, use it and say so; if you don't, still handle the pressure and tilt fields correctly and note that you could not test them.

My canvas is good but my experiment is thin. Which do I fix? The experiment. Section 5 and the report together carry 37 points — more than the canvas itself. Many candidates get this backwards.

Can I submit early? Yes, and it is genuinely a good signal. Tag the commit and send the email whenever you are done.

> ### A closing word
>
> Most take-home assignments ask you to demonstrate that you can produce a lot of code quickly. This one asks for the opposite: **restraint, measurement, and the confidence to finish one thing properly.**
>
> We would rather read a canvas that does four things beautifully, backed by a report that tells us something true about latency and cost, than a feature list we cannot verify. Build the small thing. Measure it honestly. Tell us what you found — including what didn't work.

Questions: **proscio@scholera-inc.com** · Assignment v1.0 · Reference project: github.com/penecho/penecho (AGPL-3.0), used here for study and inspiration only; Scholera is not affiliated with it.